Bitácora técnica

Entrada 2 · Fase 3: conteo, perfil corporal y conexión

Proyectos	Proyecto-de-pose (Python) · Camcel (Flutter)
Tecnologías	MediaPipe, OpenCV, CustomTkinter, Flutter, Dart, Kotlin
Período	13/09/2026 – 19/10/2026
Antecedente	Continúa la bitácora técnica del 13/09/2026
Estado	Fase 3 completada · Fase 4 en preparación

1. Resumen del trabajo

La entrada anterior cerró con la Fase 2 (ángulos y postura de espalda) y con Camcel transmitiendo video con baja latencia. Quedaban abiertos tres frentes: calibrar los umbrales con datos reales, contar repeticiones (Fase 3) y decidir desde qué ángulo se graba. En este período se cerraron los tres, y apareció un cuarto que no estaba previsto: conectar el celular al PC cuando no hay una red WiFi común.

El trabajo se agrupa así:

- **Interfaz y rendimiento** de la app de escritorio: rediseño del panel y procesado en un hilo aparte.
- **Calibración** del umbral de encorvamiento con mediciones propias.
- **Conexión por QR** y punto de acceso propio del celular.
- **Investigación de reconstrucción 3D** desde una sola cámara, con resultado negativo documentado, y la evaluación de un sensor láser.
- **Fase 3**: conteo de repeticiones, perfil corporal y filtro de detecciones incoherentes.
- **Revisión de Camcel**: tres errores corregidos y verificados en el teléfono, e ícono compartido entre las dos aplicaciones.

La decisión de diseño que estaba abierta queda cerrada: **la cámara va de perfil**. Se tomó por medición, no por preferencia (sección 5).

2. Interfaz y rendimiento de la app de escritorio

2.1 Rediseño del panel

La app se usa a varios metros de la pantalla, mientras se hace el ejercicio. Eso guió el rediseño: el veredicto de postura pasa de ser una etiqueta a un campo de color a todo el ancho, que se lee de reojo. Las medidas dejan de ser tarjetas y pasan a una hoja de datos con filetes.

- Tema oscuro con colores nombrados por función (texto, superficie, campo…). Contrastes comprobados: 12.9:1 el texto principal, 5.8:1 el secundario y 5.3:1 / 5.4:1 los campos de estado, todos por encima del mínimo de 4.5:1 de WCAG.
- Tipografía Bahnschrift, la DIN incluida en Windows. Se verifica que esté instalada en lugar de suponerlo.
- Histéresis en el veredicto: sin ella, el campo de color parpadeaba cuando el valor rondaba el umbral.

2.2 Procesado en un hilo aparte

Captura e inferencia se ejecutaban en el hilo de Tk, el mismo que atiende la interfaz. Se movieron a un hilo propio. La mejora se midió como el hueco mediano entre eventos de la interfaz:

Cambio	Antes	Después
Captura e inferencia fuera del hilo de Tk	46.1 ms	17.3 ms
Reutilizar el `PhotoImage` con `paste()` en vez de recrearlo	15 ms	8 ms
Quitar la segmentación (ponía el fondo en gris)	0.4 ms	—

2.3 Correcciones

Problema	Causa	Solución aplicada
La ventana no arrancaba maximizada.	CustomTkinter aplica su geometría en un callback posterior que deshacía `state("zoomed")`.	Maximizar desde el evento `<Map>`, cuando la ventana ya existe.
Al cambiar de fuente, los campos saltaban al final del panel.	El selector reinsertaba su contenedor al final del panel.	Cada opción alterna sus campos dentro de un contenedor fijo, que no se mueve.
El aviso de «señal perdida» no se veía.	`detener()` lo sobrescribía al instante.	El aviso se escribe después de detener, con el motivo del corte y un consejo.
La ventana se quedaba en «Conectando» para siempre.	El vigilante de inactividad no cubría la fase de conexión.	El límite de tiempo sin imágenes cubre también la conexión.
Parar y arrancar rápido dejaba dos bucles de pintado vivos.	El pintado se reprograma solo con after(); el pendiente revivía tras volver a empezar.	Cada sesión de medida lleva un número, y la cadena que no coincide se apaga.
Campo de último número vacío: error cinco segundos después culpando al celular.	Se armaba una URL inválida y se intentaba abrir.	Validar la dirección antes de conectar.

3. Calibración del umbral de encorvamiento

El umbral provisional era 0.20. Se analizó qué significa ese número: el índice vale aproximadamente $0.389 \cdot \text{sen}(\theta)$, donde θ es el adelantamiento de la cabeza respecto al tronco. 0.20 equivale a $\theta \approx 30°$, un encorvamiento severo; por eso el sistema avisaba tarde.

Se creó `calibrar_encorvamiento.py`, que mide el índice con la cámara y el cuerpo de quien va a usar la app, en postura recta y encorvada, y propone el umbral.

Fuente	Espalda recta	Encorvado	Umbral
Calibración con la cámara propia	0.024	0.140	0.08
Fotos de referencia	máx. 0.073	mín. 0.144	entre ambos

El umbral queda en **0.08** (`UMBRAL_JOROBA`), con 0.02 de histéresis. Las dos fuentes coinciden: ninguna espalda recta clara pasa de 0.073 y ninguna encorvada baja de 0.144.

4. Conexión por QR y punto de acceso

Problemática. Camcel necesita que el PC alcance la IP del celular. Con datos móviles eso es imposible: el operador asigna una dirección privada detrás de su NAT, que no es accesible desde fuera.

Solución. El celular crea su propio punto de acceso (`LocalOnlyHotspot` de Android) y muestra un QR con el nombre de la red, la clave y su dirección. El PC lee el QR con la webcam y entra a esa red solo, mediante `netsh wlan` (`conexion_qr.py`).

Hallazgo. Muchas guías dicen que el punto de acceso local siempre usa 192.168.49.1. En el Poco F5 la red sale en 10.65.x.x, indistinguible por su forma de la IP de datos móviles. La única forma fiable de encontrarla es comparar las interfaces de red antes y después de encender el punto de acceso, y tomar la que apareció.

También se añadió `FLAG_KEEP_SCREEN_ON` durante la transmisión: si la pantalla se apaga, Android suspende la cámara y el stream se corta.

5. Reconstrucción 3D desde una cámara

Antes de fijar el ángulo de grabación se investigó si se podía obtener la pose en 3D con una sola cámara. Si funcionaba, el ángulo de grabación dejaría de importar.

5.1 Método

Si se conoce la longitud real de un hueso y el rayo de cámara por el que se ve su extremo, ese extremo solo puede estar en dos puntos del espacio: es la intersección de una recta con una esfera. Las longitudes salen del perfil corporal de la persona.

Elegir entre esos dos puntos mirando un hueso aislado resultó imposible. Se probó elegir siempre el más cercano, el menos inclinado y seguirlo en el tiempo, y ninguno funcionó: las dos soluciones son continuas y parten del mismo sitio. Lo que sí desempata es la cadena completa. El pie no se despega del suelo, y solo una de las dos rodillas posibles deja el tobillo donde realmente está.

5.2 Resultado

Condición	Error medio del ángulo de rodilla
Landmarks perfectos (180° a 80°)	0.00°
0.5 px de ruido en los landmarks	≈ 30°
Láser como ancla exacta del tobillo, sin ruido de imagen	7°
Láser como ancla del tobillo, 1 px de ruido	18°

El método es exacto pero no se puede usar: MediaPipe tiene más de medio píxel de ruido. El problema está mal condicionado justo con la pierna recta, que es como empieza toda sentadilla. Además, cuando se elige mal entre los dos puntos, el error no es pequeño: duplica la inclinación. `ReconstructorPierna` queda en el código, documentado como no usable con video real. Equivocarse en el campo de visión apenas importa (10° de FOV cuestan unos 3° de ángulo): el eslabón débil es el ruido.

5.3 Sensor láser de distancia

Se simuló qué aportaría un láser de distancia. **Para el tronco es decisivo:** con dos sensores, uno arriba y otro a la altura de la cadera, la inclinación sale de la diferencia de distancias, sin pasar por la imagen. Con 1 cm de ruido de sensor, el error es de 1° entre 0° y 30°, frente a los 4–9° de estimarla por el escorzo en la imagen. **Para las piernas no sirve:** el láser resuelve dónde está el tobillo, pero la ambigüedad está en la rodilla, y ningún sensor mira ahí.

5.4 Lo que sí quedó fiable

- Las **razones** entre segmentos (fémur/tibia, tronco/fémur) no dependen de la distancia: valen 1.000 exacto a 2, 3, 4.5 y 6 m.
- Las medidas en centímetros tienen un 5 % de error sistemático. Viene de extrapolar la coronilla, que MediaPipe no marca.

Decisión: sin 3D, el ángulo de grabación sí importa, y la sentadilla se evalúa sobre todo por la rodilla y el tronco, que se ven en el plano sagital. **La cámara va de perfil.** El costo, que queda anotado, es que el valgo de rodilla (las rodillas hacia adentro) solo se ve de frente.

6. Fase 3: conteo de repeticiones

`conteo_sentadillas.py` implementa una máquina de estados: de pie → bajando → abajo → subiendo → de pie. Una repetición se cuenta solo cuando se recorre el ciclo completo, no al cruzar un umbral. El ángulo tiembla unos grados, y con un solo umbral cada repetición real sumaba varias.

Parámetro	Valor	Función
`ANGULO_DE_PIE`	160°	Por encima, la persona está de pie.
`ANGULO_INTENTO`	140°	Bajar de aquí ya cuenta como intento de sentadilla.
`ANGULO_PROFUNDIDAD`	100°	Bajar de aquí es una sentadilla con profundidad suficiente.
`MARGEN`	8°	Histéresis entre estados, contra el temblor.

Cada repetición se juzga por separado: si llegó a la profundidad y si hubo una alerta de postura *en algún momento* del recorrido, no solo al final. Una bajada corta se cuenta y se marca como tal, en vez de ignorarse: diez medias sentadillas con el contador en cero no le dicen nada al usuario. Agacharse a recoger algo, en cambio, no cuenta.

Se probó con series sintéticas: cuenta 5 de 5 incluso con 8° de ruido, muy por encima del de MediaPipe. El módulo incluye 11 pruebas automáticas (doctests).

7. Perfil corporal y filtro de coherencia

`perfil_corporal.py` mide una vez las proporciones de la persona (se ingresa la estatura y se promedian 30 fotogramas) y las guarda. Sirve para comparar contra una referencia propia en lugar de valores genéricos. La primera medición propia dio: estatura 182 cm, razón fémur/tibia 1.10 y razón tronco/fémur 1.80.

Las proporciones de una persona no cambian de un fotograma al siguiente. Eso permite un filtro (`deteccion_coherente`) que descarta los fotogramas en los que MediaPipe perdió un punto: si la razón entre dos huesos se aleja de la del perfil, la detección es errónea.

7.1 Error: el filtro solo servía de frente

La primera versión normalizaba las longitudes por el ancho de caderas. De frente, ese ancho mide unos 50 px. De perfil, las dos caderas caen casi en el mismo píxel y mide unos 2 px, así que un temblor de un píxel producía un error del 50 %. Con la cámara de perfil, el filtro habría rechazado todos los fotogramas.

Solución: comparar razones entre huesos (fémur contra tibia, tronco contra fémur) en vez de longitudes. Una razón no tiene escala: no cambia al acercarse ni al girarse. Además, cada lado del

cuerpo se comprueba por separado. Al promediar los dos hombros, perder uno solo desplazaba el centro a la mitad y el error pasaba el margen sin ser detectado. Se verificó que acepta de frente, de perfil y entre 0.6× y 1.4× de distancia, y que rechaza rodilla, tobillo, hombro y cadera perdidos.

8. Medir sobre el lado que la cámara ve

Problemática. De perfil, la pierna del lado opuesto queda tapada, y MediaPipe igual entrega sus puntos, adivinados. Los ángulos se calculaban siempre sobre el lado derecho, así que bastaba ponerse con el lado izquierdo hacia la cámara para que la sentadilla se midiera sobre una pierna invisible.

Solución. MediaPipe entrega una `visibility` por punto. Se elige el lado con mejor visibilidad, y si ninguno pasa de 0.5 no se mide: es el detector indicando que está adivinando. La medición vive en una función compartida (`medidas_de`) que usan tanto la app como el calibrador. Si midieran distinto, el umbral calibrado no valdría en la app.

9. Revisión de Camcel

Antes de la revisión, Camcel quedó bajo control de versiones (git), lo que permite volver atrás si un cambio sale mal. Todas las correcciones se verificaron en el teléfono.

Problema	Causa	Solución aplicada
Al volver a la pantalla del QR ofrecía «Crear punto de acceso» sobre una red ya creada.	El estado del punto de acceso vivía en la pantalla del QR y se perdía al salir de ella, aunque la red seguía encendida en Android.	El estado pasa a una clase (`PuntoDeAcceso`) que sobrevive a la pantalla. La principal muestra la IP del punto de acceso cuando está activo.
Tras apagar el punto de acceso, el QR seguía mostrando su dirección.	Una carrera: Android tarda unos cientos de milisegundos en desmontar la interfaz, y en ese tiempo su IP sigue apareciendo.	En vez de esperar un tiempo arbitrario, se excluye explícitamente la IP recién retirada. La pantalla principal ahora también borra una IP que ya no existe.
Al pulsar Parar con el PC conectado, la app seguía en «En el aire» y seguía transmitiendo.	El servidor esperaba a cerrar cada conexión una por una. Si el cliente no estaba leyendo, ese cierre nunca terminaba y la parada se quedaba colgada, sin error visible.	Cierre forzado del servidor (`close(force: true)`) y un límite de 2 s por paso de la parada, para que la pantalla nunca quede mostrando un estado falso.

Verificación del último caso. Durante la prueba el teléfono solo tenía datos móviles, así que el PC se conectó por el cable USB (`adb forward tcp:8080 tcp:8080`). El PC estaba recibiendo video (190 fotogramas), se pulsó Parar, el stream se cortó en el PC y la app volvió a «Cámara lista».

10. Ícono compartido

`generar_icono.py` dibuja el ícono y lo exporta para las dos aplicaciones desde una sola fuente, para que no se desincronicen. Muestra lo mismo que la app dibuja sobre el video: una figura de perfil en sentadilla, con el tronco en verde y un arco en el ángulo de rodilla. Cada tamaño de Android (48 a 192 px) se dibuja de nuevo en vez de reducir el grande, porque al reducirlo los trazos finos se emborronan.

En Windows hicieron falta dos ajustes. El primero, fijar un identificador de aplicación (AppUserModelID) antes de crear la ventana: sin él, la barra de tareas agrupa la app bajo python.exe y muestra el ícono

de Python. El segundo, aplicar el ícono justo al crear la ventana, porque CustomTkinter pone el suyo unos milisegundos después si nadie lo hizo.

11. Estado actual y próximos pasos

Completado

- Interfaz de escritorio rediseñada, con el procesado fuera del hilo de la interfaz.
- Umbral de encorvamiento calibrado con mediciones propias (0.08).
- Conexión por QR, incluso sin WiFi común, mediante el punto de acceso del celular.
- Decisión de ángulo de captura cerrada por medición: cámara de perfil.
- Conteo de repeticiones con juicio de profundidad y postura por repetición.
- Perfil corporal y filtro de detecciones incoherentes, válido de perfil.
- Medición sobre el lado visible del cuerpo.
- Camcel: parada fiable, estado del punto de acceso correcto e ícono propio.

Pendiente inmediato

- **Recalibrar el encorvamiento.** Ahora se mide con la oreja del lado visible, y el umbral 0.08 se obtuvo con el lado derecho fijo.
- **Calibrar la profundidad.** El umbral de 100° de rodilla es provisional; falta medirlo en sentadillas reales.
- **Probar en vivo la conexión automática** del PC a la red del celular leída del QR.
- **Errores de rodilla en el plano frontal** (valgo): no se ven de perfil. Requieren una segunda cámara o una toma frontal aparte.
- **Fase 4:** retroalimentación al usuario durante el ejercicio y migración de la lógica a Flutter.

Anexo: registro de cambios

Cada cambio quedó guardado en el control de versiones con una descripción del problema y de la solución.

Fecha	Proyecto	Commit	Cambio
19/09	Python	`9b838b5`	Rediseño del panel, procesado en hilo aparte y umbral calibrable
19/10	Python	`10bd2bd`	Conexión por QR, umbral calibrado y perfil corporal
19/10	Python	`537e1f5`	Reconstrucción 3D desde una cámara: exacta, pero no usable con ruido
19/10	Python	`d062141`	Medido qué aporta un láser de distancia a la reconstrucción
19/10	Python	`1b8c4b1`	Fase 3: conteo de repeticiones, perfil corporal y cámara de perfil
19/10	Python	`03ed2d3`	El filtro de coherencia servía solo de frente
19/10	Python	`00040c7`	Medir sobre el lado que la cámara ve
19/10	Python	`0978d01`	Ícono propio, el mismo que Camcel
19/10	Camcel	`a09e5f3`	Estado de Camcel antes de la revisión
19/10	Camcel	`71c179c`	Recordar el punto de acceso fuera de la pantalla del QR
19/10	Camcel	`85fe1ad`	No anunciar la IP de un punto de acceso ya apagado
19/10	Camcel	`4c1e98f`	Parar corta de verdad la transmisión, e ícono propio